

Bitácora de uso de Inteligencia Artificial

Trabajo

Materia: Proncesamiento del habla

Trabajo práctico: TP1 - PROCESAMIENTO DEL HABLA: webscrapping

Fecha: 13 de septiembre de 2026

Institución: ISSD

Docente: Ana Driedich

Cree una nube de palabras de un texto en español.

* Dicho texto puede ser descargado de Internet (webscrapping) o extraído del dataset de alguna librería.

* Puede usar cualquier librería o herramienta para obtener el texto de su interés.

* En este notebook explica el paso a paso del proceso.

Herramientas utilizadas

- Codex — modelo 5.6 Sol
- Codex

Sesión 1: Webscrapping

Fecha: 13 de septiembre de 2026

Herramienta: Codex

Modelo: 5.6 Sol

Prompt 1

Como veras estoy parado en un notebook que dice SANCHEZ_AGUSTIN_PH_TP1 para este trabajo cual crees vos que es la mejor libreria para realizar webscrapping ? y porque tu eleccion ?

RESPUESTA

Voy a revisar el notebook y, sobre todo, qué sitio, datos y restricciones plantea el TP. Con eso te recomiendo una librería concreta y te explico por qué encaja mejor que las alternativas.

Para este trabajo elegiría **BeautifulSoup** (`'bs4'`) junto con `'requests'`.

La consigna pide extraer el discurso de **una sola página web**, por lo que no necesitás una herramienta pesada como Selenium o Scrapy. La combinación sería:

- `'requests'`: descarga el HTML de la página.
- `'BeautifulSoup'`: localiza y extrae los párrafos que contienen el discurso.

¿Por qué esta elección?

- Es sencilla de aprender y explicar paso a paso.
- Funciona muy bien en Google Colab.
- Permite seleccionar elementos HTML como `'<article>'`, `'<p>'` o una clase específica.
- Te da control para evitar menús, publicidad, encabezados y otros textos que contaminarían la nube de palabras.
- El resultado queda directamente como texto de Python, listo para aplicar stopwords, separar oraciones y calcular TF-IDF.
- Es suficiente para una página estática, que sería la opción más conveniente para este TP.

Una estructura inicial sería:

```
``python
import requests
from bs4 import BeautifulSoup

url = "URL_DEL_DISCURSO"

headers = {
    "User-Agent": "Mozilla/5.0"
}

respuesta = requests.get(url, headers=headers, timeout=15)
respuesta.raise_for_status()

soup = BeautifulSoup(respuesta.text, "html.parser")
```

```
# Este selector debe adaptarse a la estructura de la página elegida
```

```
parrafos = soup.select("article p")
```

```
texto = " ".join(  
    parrafo.get_text(" ", strip=True)  
    for parrafo in parrafos  
)
```

```
print(texto[:1000])
```

```
``
```

Lo más importante es elegir una web donde el discurso aparezca directamente en el HTML. Si el contenido solamente aparece después de ejecutar JavaScript, 'requests' no podrá verlo y habría que recurrir a Selenium o Playwright. Para este trabajo evitaría ese tipo de página, porque agrega complejidad que no aporta demasiado al objetivo principal, que es el procesamiento del texto.

Mi justificación para colocar en el notebook sería:

> Se eligió BeautifulSoup junto con Requests porque el trabajo requiere extraer texto de una única página web estática. Requests permite descargar su código HTML, mientras que BeautifulSoup facilita identificar y extraer únicamente los párrafos correspondientes al discurso. Esta combinación es liviana, fácil de utilizar en Google Colab y permite obtener un texto limpio que luego puede procesarse para eliminar stopwords, construir la matriz documento-vocabulario, calcular TF-IDF y generar la nube de palabras.

Por tanto, mi elección concreta es **BeautifulSoup**, usando 'requests' como complemento para descargar la página.

Sesión 2: Recuperar texto de la página

Fecha: 17 de septiembre de 2026

Herramienta: Codex

Importado desde codex. Solo contiene lo enviado; no se inventó contenido.

Prompt 1

Bueno continuamos con el trabajo practico deberia recuperar todo el texto. que se encuentra en la pagina y meterlo todo en una variable o como seria la forma mas optima de hacerlo ? Charlemos no ejecutes nada

RESPUESTA

Sí, charlemos. No voy a ejecutar ni modificar nada.

Si necesitas recuperar **todo el texto de una página web**, la forma óptima depende de qué quieras hacer después. En general, no conviene meter absolutamente todo en una única variable sin estructura, porque perderías qué texto corresponde al título, a cada párrafo, enlace, sección, etc.

Hay tres enfoques habituales:

1. **Texto plano completo**

```
```js
const textoCompleto = document.body.innerText;
```
```

Esto obtiene el texto visible de la página. Es útil para búsquedas, análisis o una entrega simple. Mantiene saltos de línea razonables y excluye etiquetas HTML, scripts y estilos.

2. **Contenido HTML completo**

```
```js
const contenidoHTML = document.body.innerHTML;
```
```

Conserva etiquetas y estructura, pero también introduce bastante contenido que quizá no necesitas. Sirve si después quieres reconstruir o procesar el documento.

3. **Información estructurada – normalmente la mejor opción**

```
```js
const contenido = {
 titulo: document.querySelector("h1")?.innerText ?? "",
 subtitulos: [...document.querySelectorAll("h2")].map(
 elemento => elemento.innerText
),
 parrafos: [...document.querySelectorAll("p")].map(
 elemento => elemento.innerText
)
};
```

```

),
 enlaces: [...document.querySelectorAll("a")].map(enlace => ({
 texto: enlace.innerText,
 url: enlace.href
 })))
};
```

```

Esta opción permite acceder después a `contenido.titulo`, `contenido.parrafos`, etc., sin tener que separar nuevamente una gran cadena.

Mi recomendación para el trabajo práctico sería combinar ambos:

```

```js
const textoCompleto = document.body.innerText;

const contenido = {
 titulo: document.querySelector("h1")?.innerText ?? "",
 parrafos: [...document.querySelectorAll("p")].map(p => p.innerText),
 textoCompleto
};
```

```

Una precisión importante: `document.body.innerText` recupera el texto de la página **que ya está cargado en el navegador**. Si el texto se obtiene dinámicamente mediante una API, quizá resulte mejor guardar directamente la respuesta de esa API en un objeto, en vez de volver a leerla desde el DOM.

En resumen: si únicamente te piden “recuperar todo el texto”, `document.body.innerText` es suficiente. Si después hay que clasificarlo, mostrar ciertas partes o enviarlo a otro lugar, un objeto estructurado es una solución más limpia.

Prompt 2

Buenísimo como lo aplico en python con la estructura que ya estoy utilizando en este tp1 de ph?

RESPUESTA

Perfecto. Sin ejecutar ni modificar nada: en Python depende de cómo estés accediendo a la página en el TP.

Si ya utilizás **Selenium**, el equivalente a `document.body.innerText` sería:

```

```python
from selenium.webdriver.common.by import By

texto_completo = driver.find_element(By.TAG_NAME, "body").text
```

```

Podrías integrarlo en una función para mantener ordenada la estructura:

```

```python

```

```
from selenium.webdriver.common.by import By
```

```
def obtener_texto_pagina(driver):
 cuerpo = driver.find_element(By.TAG_NAME, "body")
 return cuerpo.text
...
```

Y usarlo así:

```
```python  
texto_completo = obtener_texto_pagina(driver)  
  
print(texto_completo)  
...
```

Si necesitás una estructura más organizada:

```
```python  
from selenium.webdriver.common.by import By

def obtener_contenido(driver):
 titulo = driver.title
 texto_completo = driver.find_element(By.TAG_NAME, "body").text

 parrafos = [
 elemento.text
 for elemento in driver.find_elements(By.TAG_NAME, "p")
 if elemento.text.strip()
]

 return {
 "titulo": titulo,
 "parrafos": parrafos,
 "texto_completo": texto_completo
 }
...
```

Después accederías a los datos así:

```
```python  
contenido = obtener_contenido(driver)  
  
print(contenido["titulo"])  
print(contenido["texto_completo"])  
...
```

Si en cambio estás usando **Requests** y **BeautifulSoup**, sería:

```
``python
import requests
from bs4 import BeautifulSoup

respuesta = requests.get("URL_DE_LA_PAGINA")
soup = BeautifulSoup(respuesta.text, "html.parser")

texto_completo = soup.get_text(separator="\n", strip=True)
``
```

La diferencia importante es:

- **Selenium** obtiene el texto que aparece después de cargar y ejecutar JavaScript.
- **BeautifulSoup** procesa el HTML que devuelve inicialmente el servidor.
- Si la página carga contenido dinámicamente, Selenium suele ser la opción adecuada.

Para adaptarlo exactamente a la estructura de tu TP1, pegame el archivo principal o la parte del código donde abris la página. Con eso te indico en qué función y lugar debería entrar, sin ejecutar ni cambiar nada.

Prompt 3

Estoy usando beautiful soap ,

```
import requests
from bs4 import BeautifulSoup
```

```
url = "[https://larevuelta.com.ar/2026/09/05/olvidense-de-los-pechos/](https://larevuelta.com.ar/2026/09/05/olvidense-de-los-pechos/)"
```

```
respuesta = requests.get(url)
respuesta.raise_for_status()
```

```
soup = BeautifulSoup(respuesta.text, 'html.parser')
```

```
titulo = soup.find('h1', *class_='page-title')
if titulo:
    print("Titulo del artículo:", titulo.get_text())
```

```
else:
    print("No se encontró el título del artículo.")
```

RESPUESTA

Perfecto, entonces estás bien encaminado con BeautifulSoup. Hay dos detalles en el código pegado que parecen venir del formato de Markdown:

- La URL debe contener solamente la dirección, sin `'[texto](dirección)'`.
- En Python se escribe `'class_'`, `'raise_for_status()'` y `'get_text()'` sin barras invertidas.

Para recuperar todo el texto, podrías ampliar tu estructura así:

```
```python
import requests
from bs4 import BeautifulSoup

url = "https://larevuelta.com.ar/2026/09/05/olvidense-de-los-pechos/"

respuesta = requests.get(url, timeout=15)
respuesta.raise_for_status()

soup = BeautifulSoup(respuesta.text, "html.parser")

titulo = soup.find("h1", class_="page-title")

if titulo:
 print("Título del artículo:", titulo.get_text(strip=True))
else:
 print("No se encontró el título del artículo.")

texto_completo = soup.get_text(separator="\n", strip=True)

print(texto_completo)
```
```

Pero `soup.get\_text()` recupera también el menú, encabezado, pie de página y otros textos. Para un artículo periodístico, resulta mejor localizar primero el bloque `

`:

```
```python
articulo = soup.find("article")

if articulo:
 texto_articulo = articulo.get_text(separator="\n", strip=True)
 print(texto_articulo)
else:
 print("No se encontró el artículo.")
```
```

Una opción aún más limpia es recuperar el título y los párrafos por separado:

```
```python
articulo = soup.find("article")

if articulo:
 titulo = articulo.find("h1")
```

```

 parrafos = [
 parrafo.get_text(" ", strip=True)
 for parrafo in articulo.find_all("p")
 if parrafo.get_text(strip=True)
]

 contenido = {
 "titulo": titulo.get_text(" ", strip=True) if titulo else "",
 "parrafos": parrafos,
 "texto_completo": "\n\n".join(parrafos)
 }

 print(contenido["titulo"])
 print(contenido["texto_completo"])
else:
 print("No se encontró el artículo.")
'''

```

Esta última sería mi recomendación para el TP porque `contenido` conserva la información ordenada:

```

'''python
contenido["titulo"]
contenido["parrafos"]
contenido["texto_completo"]
'''

```

También evitaría depender inicialmente de `class="page-title"`, porque la clase puede variar. Buscar el `

` y luego su `

# ` relaciona mejor el título con la noticia. No ejecuté ni modifiqué nada.

## Prompt 4

Relizamos un markdown explicativo encima de ese bloque de código está marcado

### RESPUESTA

Sí, encima de ese bloque de código podés agregar una celda Markdown con esta explicación:

```

'''markdown
Extracción del contenido del artículo

```

En este bloque se realiza una petición HTTP a la URL del artículo mediante la biblioteca `requests`. Luego, se comprueba que la respuesta del servidor sea correcta utilizando `raise\_for\_status()`.

El código HTML obtenido se procesa con `BeautifulSoup`, lo que permite buscar y extraer elementos específicos de la página. Primero se localiza el artículo mediante la etiqueta `

` y después se recuperan su título y sus párrafos.

Finalmente, la información se almacena en un diccionario llamado `contenido`, compuesto por:

```
- 'titulo': titulo principal del articulo.
- 'parrafos': lista con los párrafos encontrados.
- 'texto_completo': unión de todos los párrafos en una sola cadena de texto.
...
```

Debajo iría la celda de código:

```
```python
import requests
from bs4 import BeautifulSoup

url = "https://larevuelta.com.ar/2026/09/05/olvidense-de-los-pechos/"

respuesta = requests.get(url, timeout=15)
respuesta.raise_for_status()

soup = BeautifulSoup(respuesta.text, "html.parser")

articulo = soup.find("article")

if articulo:
    titulo = articulo.find("h1")

    parrafos = [
        parrafo.get_text(" ", strip=True)
        for parrafo in articulo.find_all("p")
        if parrafo.get_text(strip=True)
    ]

    contenido = {
        "titulo": titulo.get_text(" ", strip=True) if titulo else "",
        "parrafos": parrafos,
        "texto_completo": "\n\n".join(parrafos)
    }

    print(contenido["titulo"])
    print(contenido["texto_completo"])
else:
    print("No se encontró el articulo.")
...
```
```

La explicación deja claro el objetivo del bloque, las bibliotecas utilizadas y la estructura en la que se guarda la información.

Buenísimo buena implementación , me explicas un poco paso a paso como funciona el código ?

## RESPUESTA

Claro. El código sigue cuatro etapas: descarga la página, interpreta su HTML, extrae el artículo y organiza el resultado.

### ### 1. Importación de bibliotecas

```
```python
import requests
from bs4 import BeautifulSoup
```
```

- 'requests' permite hacer una petición a una página web y descargar su HTML.
- 'BeautifulSoup' permite recorrer y buscar elementos dentro de ese HTML.

### ### 2. Descarga de la página

```
```python
url = "https://larevuelta.com.ar/2026/09/05/olvidense-de-los-pechos/"

respuesta = requests.get(url, timeout=15)
respuesta.raise_for_status()
```
```

La URL se guarda en una variable para poder reutilizarla.

```
```python
requests.get(url, timeout=15)
```
```

Realiza una petición HTTP de tipo 'GET'. La respuesta se guarda en 'respuesta'.

El 'timeout=15' establece un máximo de 15 segundos de espera. Así, el programa no queda esperando indefinidamente si la página no responde.

```
```python
respuesta.raise_for_status()
```
```

Comprueba el código de estado HTTP:

- '200': la página se obtuvo correctamente.
- '404': la página no existe.
- '500': ocurrió un error en el servidor.

Si recibe un código de error, genera una excepción y evita continuar trabajando con una respuesta inválida.

### ### 3. Interpretación del HTML

```
```python
soup = BeautifulSoup(respuesta.text, "html.parser")
```
```

'respuesta.text' contiene el HTML descargado como una cadena de texto.

BeautifulSoup interpreta ese texto y crea un objeto llamado 'soup'. Desde ese objeto podemos buscar etiquetas como '<article>', '<h1>' y '<p>'.

```
```python
articulo = soup.find("article")
```
```

'find()' busca la primera etiqueta '<article>' de la página.

Normalmente, esta etiqueta contiene el cuerpo principal de una noticia. Buscar dentro de ella evita recuperar textos del menú, encabezado o pie de página.

### ### 4. Verificación del artículo

```
```python
if articulo:
```
```

Comprueba si BeautifulSoup encontró la etiqueta '<article>'.

- Si la encontró, 'articulo' contiene el elemento HTML.
- Si no la encontró, su valor será 'None' y se ejecutará el 'else'.

### ### 5. Extracción del título

```
```python
titulo = articulo.find("h1")
```
```

Busca el primer encabezado '<h1>' dentro del artículo. Generalmente corresponde al título principal de la noticia.

Todavía no se extrae el texto: 'titulo' contiene la etiqueta completa.

Por ejemplo:

```
```html
<h1>Olvidense de los pechos</h1>
```
```

### ### 6. Extracción de los párrafos

```

'''python
parrafos = [
 parrafo.get_text(" ", strip=True)
 for parrafo in articulo.find_all("p")
 if parrafo.get_text(strip=True)
]
'''

```

Esta es una comprensión de listas. Puede entenderse como una versión resumida de:

```

'''python
parrafos = []

for parrafo in articulo.find_all("p"):
 texto = parrafo.get_text(" ", strip=True)

 if texto:
 parrafos.append(texto)
'''

```

Sus partes realizan lo siguiente:

```

'''python
articulo.find_all("p")
'''

```

Busca todas las etiquetas `

` dentro del artículo.

```

'''python
parrafo.get_text(" ", strip=True)
'''

```

Extrae solamente el texto del párrafo:

- `" "` separa con espacios el contenido de etiquetas internas.
- `strip=True` elimina espacios y saltos de línea innecesarios al principio y al final.

```

'''python
if parrafo.get_text(strip=True)
'''

```

Evita agregar párrafos vacíos a la lista.

Al finalizar, `parrafos` tiene una estructura similar a:

```
```python
[
    "Primer párrafo del artículo.",
    "Segundo párrafo del artículo.",
    "Tercer párrafo del artículo."
]
```
```

### 7. Organización de los datos

```
```python
contenido = {
    "titulo": titulo.get_text(" ", strip=True) if titulo else "",
    "parrafos": parrafos,
    "texto_completo": "\n\n".join(parrafos)
}
```
```

Se crea un diccionario con tres elementos.

```
```python
"titulo": titulo.get_text(" ", strip=True) if titulo else ""
```
```

Si se encontró el `

# `, extrae su texto. Si no se encontró, guarda una cadena vacía. Esto evita un error al intentar usar `get\_text()` sobre `None`.

```
```python
"parrafos": parrafos
```
```

Guarda la lista de párrafos por separado.

```
```python
"texto_completo": "\n\n".join(parrafos)
```
```

Une todos los elementos de la lista en una sola cadena. `"\n\n"` coloca dos saltos de línea entre cada párrafo.

### 8. Presentación del resultado

```
```python
print(contenido["titulo"])
print(contenido["texto_completo"])
```
```

Accede a los valores del diccionario mediante sus claves y los muestra en la consola.

Si no se encontró la etiqueta '<article>', se ejecuta:

```
```python
else:
    print("No se encontró el artículo.")
```
```

El recorrido general sería:

```
```text
URL
↓
requests descarga el HTML
↓
BeautifulSoup interpreta el HTML
↓
Se busca la etiqueta <article>
↓
Se extraen el <h1> y los <p>
↓
Se guardan en el diccionario contenido
↓
Se muestran el título y el texto
```
```

### Sesión 3: Continuar notebook SANCHEZ\_AGUSTIN

**Fecha:** 19 de septiembre de 2026

**Herramienta:** Codex

Importado desde codex. Solo contiene lo enviado; no se inventó contenido.

---

#### Prompt 1

Buenas tardes, sigo con mi trabajo en el notebook SANCHEZ\\_AGUSTIN\\_PH\\_TP1

#### RESPUESTA

Buenas tardes. Voy a ubicar el notebook y revisar su estado para retomar desde donde quedó.

Perfecto, ya ubiqué el notebook. Está avanzado hasta la extracción del artículo y quedan por completar:

- Mostrar las stopwords del texto.
- Limpieza y nube de palabras.
- Matriz documento-vocabulario.
- Tamaño del vocabulario.
- TF-IDF y explicación/conclusiones.

También veo algunos caracteres mal codificados (por ejemplo, 'AgustÃn'), que conviene corregir antes de la entrega. Decime con cuál punto seguimos y lo armamos.

---

#### Prompt 2

from spacy.lang.es.stop\\_words import STOP\\_WORDS  
queria saber que hace esta libreria ?

#### RESPUESTA

Esa línea importa las “stop words” predeterminadas en español de spaCy:

```
```python  
from spacy.lang.es.stop_words import STOP_WORDS  
```
```

'STOP\_WORDS' es un conjunto ('set') de palabras muy frecuentes que normalmente aportan poco significado al análisis textual, como:

```
```python  
"el", "la", "de", "que", "y", "en", "para", "con"  
```
```

Podés verlo así:

```
```python  
print(STOP_WORDS)
```

```
'''
```

O comprobar una palabra:

```
```python
"el" in STOP_WORDS # True
"pechos" in STOP_WORDS # False
'''
```

En tu trabajo sirve para eliminar esas palabras antes de generar la nube de palabras o crear la matriz documento-vocabulario. spaCy no se instala con esa línea: es una lista incluida dentro de la librería spaCy.

---

### Prompt 3

Y que diferencia hay con hacer esto&#x20;

```
import spacy

Cargar el modelo en español

nlp = spacy.load('es\core\news_sm')

texto_espanol = "Los estudiantes están aprendiendo procesamiento de lenguaje natural en la clase de hoy."

Procesar el texto

doc = nlp(texto_espanol)

Realizar el preprocesamiento con una comprensión de lista

Filtramos por stop words y puntuación, y obtenemos el lema

limpieza_spacy = [
{
"Original": token.text,
"Lema": token.lemma__,
"Es_Stopword": token.is_stop
}
for token in doc if not token.is_punct
]

print(f"Texto Original: {texto_espanol}\n")
print("Resultado detallado con SpaCy:")
for item in limpieza_spacy:
print(f"Palabra: {item['Original'][:15]} | Lema: {item['Lema'][:15]} | ¿Es Stopword?: {item['Es_Stopword']}")
```

## RESPUESTA

La diferencia principal es el alcance:

```
```python
from spacy.lang.es.stop_words import STOP_WORDS
```
```

solo importa el listado de stopwords en español. Es rápido y no requiere cargar un modelo lingüístico. Sirve si únicamente querés decidir si una palabra frecuente debe excluirse:

```
```python
palabras_limpias = [p for p in palabras if p.lower() not in STOP_WORDS]
```
```

En cambio:

```
```python
nlp = spacy.load("es_core_news_sm")
doc = nlp(texto_espanol)
```
```

carga un modelo completo de español y analiza el texto. Por cada token puede darte:

- `'token.text'`: palabra original.
- `'token.lemma_'`: lema, por ejemplo, “están” → “estar”.
- `'token.is_stop'`: si spaCy la considera stopwords.
- `'token.is_punct'`: si es puntuación.
- Además, categoría gramatical, dependencias sintácticas, entidades, etc.

Tu código actual no elimina stopwords: las identifica y las muestra porque solo excluis puntuación:

```
```python
for token in doc if not token.is_punct
```
```

Para quitarlas realmente y conservar los lemas, sería:

```
```python
limpieza_spacy = [
    token.lemma_.lower()
    for token in doc
    if not token.is_stop and not token.is_punct and not token.is_space
]

print(limpieza_spacy)
```
```

Para el TP, spaCy es una buena opción porque podés justificar que, además de quitar stopwords y puntuación, normalizaste las palabras mediante lematización.

---

#### Prompt 4

```
##Saco los parrafos que no quiero que aparezcan en el resumen, en este caso el primero y el noveno , ya que los mismos son irrelevantes*
```

```
parrafos_filtrados = [p for i, p in enumerate(contenido["parrafos"]) if i not in [0, 8]]
```

```
doc = nlp(" ".join(parrafos_filtrados))
```

```
limpieza_spacy = [
```

```
 {
 "original": token.text,
 "lemma": token.lemma_ ,
 "es_stopword": token.is_stop,
 }
```

```
 for token in doc if not token.is_punct
```

```
]
```

```
print(f"texto original: {" ".join(parrafos_filtrados)}")
```

```
print("Resultado detallado con SpaCy:")
```

```
limpieza_df = pd.DataFrame(limpieza_spacy)
```

```
limpieza_df
```

que opinas de esta resolucion , es mejor para el trabajo practico , le agregarias algo mas ?

#### RESPUESTA

Es una muy buena resolución para el punto 2.1: mostrarás qué palabras spaCy reconoce como stopwords y, al mismo tiempo, dejás evidencia del lematizado. Además, `enumerate` empieza en 0, así que `[0, 8]` efectivamente excluye el primer y el noveno párrafo.

Solo ajustaría tres cosas:

1. Asegurate de importar pandas y cargar el modelo antes.

2. Tu `print` tiene un error de comillas dentro del f-string.

3. Esta celda identifica stopwords, pero todavía no las elimina; eso está bien para 2.1. En la siguiente celda las quitás para construir la nube.

Una versión prolija sería:

```
```python
import pandas as pd
import spacy
```

```

nlp = spacy.load("es_core_news_sm")

# Se excluyen el primer y el noveno párrafo, considerados irrelevantes
# para el análisis del contenido principal.
parrafos_filtrados = [
    parrafo
    for i, parrafo in enumerate(contenido["parrafos"])
    if i not in [0, 8]
]

texto_filtrado = " ".join(parrafos_filtrados)
doc = nlp(texto_filtrado)

# Se conservan los datos de cada palabra, excepto los signos de puntuación.
limpieza_spacy = [
    {
        "original": token.text,
        "lema": token.lemma_,
        "es_stopword": token.is_stop,
    }
    for token in doc
    if not token.is_punct and not token.is_space
]

limpieza_df = pd.DataFrame(limpieza_spacy)

print(f"Texto original:\n{texto_filtrado}\n")
print("Resultado detallado con spaCy:")

limpieza_df
````

```

Y para el punto siguiente, la limpieza real puede ser:

```

``python
palabras_limpias = [
 token.lemma_.lower()
 for token in doc
 if not token.is_stop
 and not token.is_punct
 and not token.is_space
 and token.is_alpha
]

texto_limpio = " ".join(palabras_limpias)

```

```
print(texto_limpio)
'''

'token.is_alpha' evita que números, símbolos u otros caracteres no alfabéticos lleguen a la nube de palabras.
```

---

## Prompt 5

De igual manera me gustaria saber que hace el

```
limpieza__spacy = [
{
"original": token.text,
"lemma": token.lemma_,
"es_stopword": token.is_stop,
}
'''

for token in doc if not token.is_punct
'''

]
```

## RESPUESTA

Ese bloque recorre cada token (palabra) de 'doc', descarta los signos de puntuación y guarda información de cada palabra en una lista de diccionarios.

```
'''python
limpieza_spacy = [
 {
 "original": token.text,
 "lemma": token.lemma_,
 "es_stopword": token.is_stop,
 }
 for token in doc
 if not token.is_punct
]
'''
```

Equivale, en forma más extensa, a:

```
'''python
limpieza_spacy = []

for token in doc:
 if not token.is_punct:
 palabra = {
 "original": token.text,
```

```

 "lemma": token.lemma_,
 "es_stopword": token.is_stop,
 }
 limpieza_spacy.append(palabra)
'''

```

Para cada palabra guarda:

- `'token.text'`: la forma original. Ej.: `'"estudiantes"'`.
- `'token.lemma_'`: su forma base o lema. Ej.: `'"estudiantes"' → '"estudiante"'`; `'"están"' → '"estar"'`.
- `'token.is_stop'`: `'True'` si es una stopwords, como “los”, “de” o “en”; `'False'` si aporta contenido, como “estudiante” o “procesamiento”.

La parte:

```

'''python
for token in doc if not token.is_punct
'''

```

significa: “para cada token de `'doc'`, siempre que no sea puntuación”. Por eso no aparecen puntos, comas, comillas, etc.

Importante: pese a llamarse `'limpieza_spacy'`, este código todavía no borra las stopwords; solo indica cuáles lo son. Para eliminarlas, cambiá la condición por:

```

'''python
for token in doc
if not token.is_punct and not token.is_stop and not token.is_space
'''

```

---

## Prompt 6

```

'''python
palabras_limpias = [
 token.lemma_.lower()
 for token in doc
 if not token.is_stop
 and not token.is_punct
 and not token.is_space
 and token.is_alpha
]

texto_limpio = " ".join(palabras_limpias)

print(texto_limpio)
'''

```

Esto seria medio que la resoulcion al 2.2 , la pregunta es en este caso que hace cada linea de este codigo , porque hacemos token, lema.lower? me explicas como si tuviera 5 años

## RESPUESTA

Si: este código es la parte central de la resolución del punto 2.2. Prepara el texto para que la nube de palabras conserve solo las palabras importantes.

Pensalo como limpiar una caja llena de palabras antes de elegir cuáles van a una cartelera.

```
``python
palabras_limpias = [
 ``
```

Creamos una lista vacía donde vamos a guardar las palabras que si queremos conservar.

```
``python
 token.lemma_.lower()
 ``
```

Por cada palabra buena, guardamos su “nombre base” en minúsculas.

- ‘token’ es una palabra que spaCy encontró en el texto. Por ejemplo: ‘"Mujeres"’.
- ‘token.lemma\_’ es su forma de diccionario o lema. Por ejemplo:
  - ‘"mujeres"’ → ‘"mujer"’
  - ‘"pensaba"’ → ‘"pensar"’
  - ‘"dijeron"’ → ‘"decir"’
- ‘.lower()’ la transforma a minúsculas:
  - ‘"Mujer"’ → ‘"mujer"’

Esto evita contar como palabras diferentes a ‘"Mujer"’, ‘"mujer"’ y ‘"MUJER"’.

```
``python
 for token in doc
 ``
```

Significa: “mirá una por una todas las palabras del texto”.

```
``python
 if not token.is_stop
 ``
```

Conservamos la palabra solo si no es una stopword. Sacamos palabras muy comunes, como ‘"la"’, ‘"de"’, ‘"y"’ o ‘"para"’.

```
``python
 and not token.is_punct
 ``
```

También quitamos signos de puntuación: ‘.’, ‘,’, ‘¿’, ‘!’, etc.

```
``python
```

```
 and not token.is_space
 ...
```

Ignoramos espacios, saltos de línea y separaciones vacías.

```
'''python
 and token.is_alpha
...'''
```

Conservamos solo elementos formados por letras. Así evitamos números, porcentajes, emojis, símbolos o combinaciones extrañas.

Por ejemplo, si el texto fuera:

```
'''python
texto = "Las mujeres dijeron: ¡Queremos ser escuchadas en 2026!"
...'''
```

Después de limpiar, podría quedar:

```
'''python
["mujer", "decir", "querer", "escuchar"]
...'''
```

Finalmente:

```
'''python
texto_limpio = " ".join(palabras_limpias)
...'''
```

Une nuevamente esas palabras usando un espacio entre cada una:

```
'''python
"mujer decir querer escuchar"
...'''
```

Y:

```
'''python
print(texto_limpio)
...'''
```

muestra el resultado para que puedas revisar qué texto se enviará a la nube de palabras.